



Sistem Parcare

Name:Campean Bogdan

Group:7

Table of Contents

<i>Deliverable 1</i>	3
Project Specification	3
Functional Requirements	3
Use Case Model	3
Use Cases Identification:.....	3
UML Use Case Diagrams	3
Supplementary Specification	4
Non-functional Requirements.....	4
Design Constraints	5
Glossary	5
<i>Deliverable 2</i>	5
Domain Model	5
Architectural Design	6
Conceptual Architecture	6
Package Design.....	7
Component and Deployment Diagram	7
<i>Deliverable 3</i>	8
Design Model	8
Dynamic Behavior	8
Class Diagram	9
Data Model	10
<i>System Testing</i>	10
<i>Future Improvements</i>	11
<i>Conclusion</i>	11
<i>Bibliography</i>	12

Deliverable 1

Project Specification

Un sistem de gestionare a unei parcare care permite adiministrarea vehiculelor, efectuarea de rezervari pe o perioada dorita selectand orice loc de parcare liber.

Functional Requirements

Sistemul trebuie sa permita adaugarea/editarea/stergera masinilor.

Sistemul trebuie sa gestioneze statusul locurilor de parcare (Liber/Ocupat)

Utilizatorul trebuie sa poata crea o rezervare legand un vehicul de un loc disponibil.

Use Case Model

Use Cases Identification:

1. Use-Case: Inregistrare Vehicul Nou

Level: User Goal

Primary Actor: Administrator sistem

Main success scenario:

1. Administratorul acceseaza pagina "Masini".
2. Apasa butonul pentru adaugarea unui vehicul nou (deschide modalul).
3. Introduce datele vehiculului (numar inmatriculare, model, culoare).
4. Apasa butonul "Salveaza".
5. Sistemul valideaza datele, salveaza vehiculul in baza de date si reimprospateaza lista.

Extensions: Datele introduse sunt incomplete: Sistemul afiseaza un mesaj de eroare si cere completarea campurilor obligatorii.

2. Use-Case: Creare Rezervare Loc Parcare

Level: User Goal

Primary Actor: Administrator / Utilizator

Main success scenario:

1. Utilizatorul acceseaza sectiunea de Rezervari.
2. Selecteaza un vehicul existent din lista derulanta.
3. Selecteaza un loc de parcare disponibil (status "Liber").
4. Selecteaza data de inceput si data de sfarsit.
5. Confirma rezervarea.
6. Sistemul calculeaza costul, salveaza rezervarea si schimba statusul locului de parcare in "Ocupat".

Extensions: Datele introduse sunt incomplete: Sistemul afiseaza un mesaj de eroare si cere completarea campurilor obligatorii.

3. Use-Case: Modificare Detalii Loc Parcare

Level: User Goal

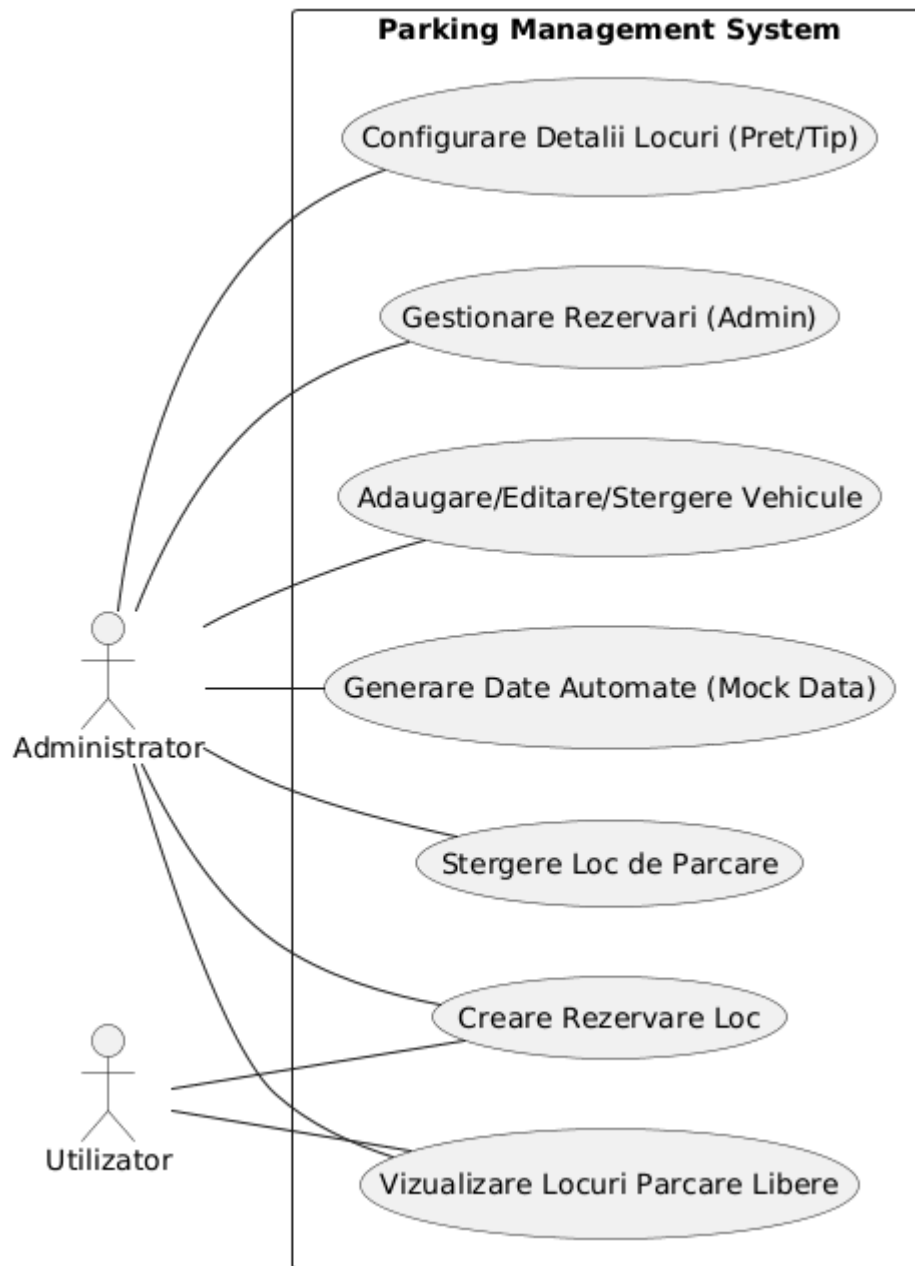
Primary Actor: Administrator sistem

Main success scenario:

1. Administratorul acceseaza lista locurilor de parcare ("Configurare Locuri Parcare").
2. Apasa butonul "Edit" pentru un anumit loc.
3. Modifica pretul pe ora sau tipul locului (ex: din Normal in Electric).
4. Salveaza modificarile.
5. Sistemul actualizeaza datele si afiseaza noul pret in lista.

Extensions: 2a. Locul este marcat ca "Ocupat": Sistemul poate restrictiona anumite modificari pana la eliberarea locului.

UML Use Case Diagrams



Supplementary Specification

Non-functional Requirements

- 1) Complexitatea redusa: Interfata utilizatorului trebuie sa fie intuitiva si usor de utilizat fiind adaptata automat la diferite dimensiuni de ecran(desktop,telefon mobil)
- 2) Securitatea: Protectia datelor vehiculelor si prevenirea modificarilor neautorizate asupra preturilor parcarii pentru a asigura increderea utilizatorilor
- 3) Fiabilitate: Sistemul trebuie sa asigure integritatea datelor la nivelul bazei de date.

- 4) Performanta: Sistemul trebuie sa asigure un timp de raspuns scurt pentru a imbunatatii experienta utilizatorilor

Design Constraints

Limbaj de programare: Java

Framework principal: Spring Boot 3.x

Gestionare dependente: Maven pentru managementul bibliotecilor externe

Arhitectura: Sistemul respecta modelul MVC (Model-View-Controller)

Interfata (Frontend): Thymeleaf pentru randarea dinamica si Bootstrap 5 pentru design

Glossary

Vehicle (Vehicul): Identificata prin model, culoare si un numar de inmatriculare unic.

License Plate (Numar Inmatriculare): Format (ex: B-123-ABC) si este obligatoriu pentru orice rezervare.

Parking Spot (Loc de Parcare): Spatiu fizic in parcare

Spot Status (Status Loc): Indicator al disponibilitatii unui loc.

Reservation (Rezervare): Leaga un vehicul de un loc de parcare pe o anumita durata.

ISO Date Time: Formatul utilizat pentru stocarea si prelucrarea datelor calendaristice (ex: YYYY-MM-DDTHH:mm).

Total Cost (Cost Total): Valoare calculata automat pe baza duratei rezervarii si al pretului locului respectiv de parcare

Deliverable 2

Domain Model

User – reprezinta un utilizator al sistemului cu rolul ADMIN sau CLIENT, avand attribute precum username, email, parola si o lista de vehicule asociate.

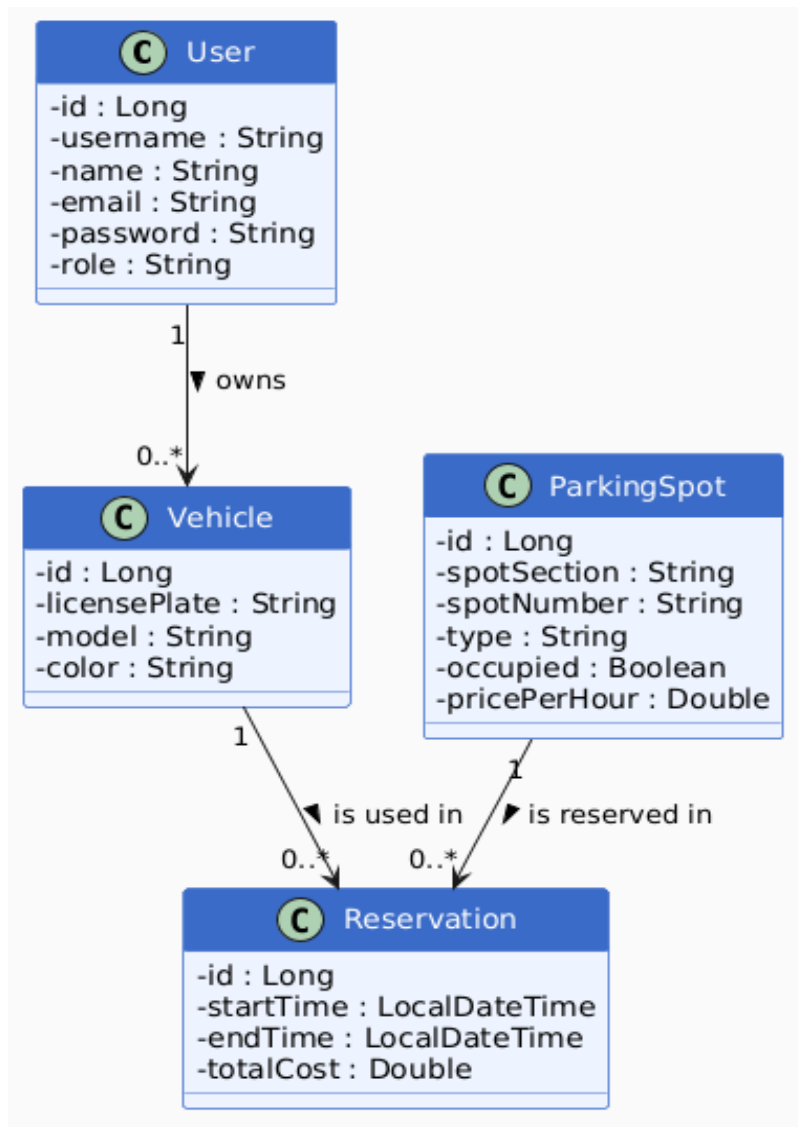
Vehicle – reprezinta un autoturism inregistrat in sistem, identificat prin numarul de inmatriculare, model si culoare. Fiecare vehicul apartine unui utilizator (ownerId).

ParkingSpot – reprezinta un loc de parcare cu attribute precum sectiunea, numarul locului, tipul (Standard/VIP), starea de ocupare si pretul pe ora.

Reservation – leaga un vehicul de un loc de parcare pe un interval de timp (startTime, endTime), avand calculat automat costul total.

****Relatii dintre entitati:****

- Un User poate detine mai multe Vehicle (relatia 1..N intre User si Vehicle, prin campul ownerId din Vehicle).
- Un Reservation este asociata unui singur Vehicle si unui singur ParkingSpot (relatii N:1).
- Un ParkingSpot poate aparea in mai multe Reservation de-a lungul timpului, dar la un moment dat poate fi ocupat de cel mult o rezervare activa.



Architectural Design

Conceptual Architecture

Architectural Style: Layered Architecture (3-Tier)

Sistemul adopta o arhitectura pe niveluri (Layered Architecture) :

- **Presentation Layer** – aplicatie React.js (SPA) care ruleaza in browser pe portul 3000. Gestioneaza interfața utilizatorului si trimite cereri HTTP catre backend.
- **Business Logic Layer** – aplicatie Spring Boot (Java) pe portul 8080. Contine logica de business organizata in controllere, servicii si repository-uri, expusa prin REST API.
- **Data Layer** – persistenta hibrida: serializare Java pe fisier (users.txt) pentru utilizatori si stocare in memorie (ArrayList) pentru rezervari, vehicule si locuri de parcare.

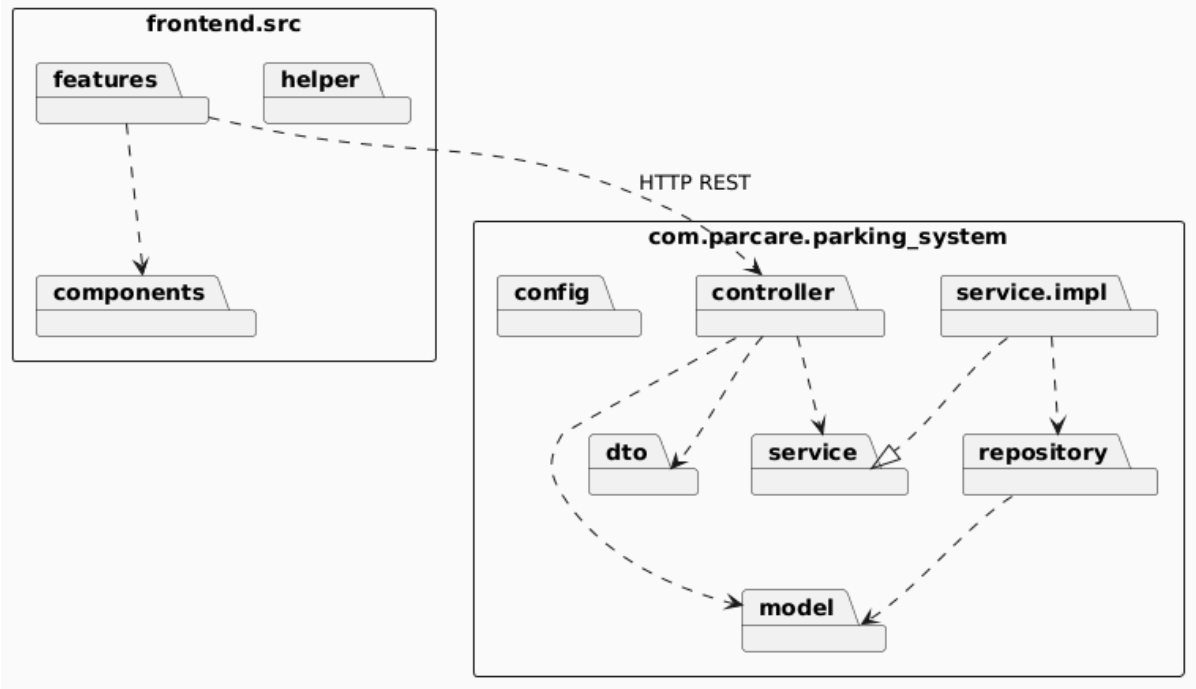
Architectural Pattern: MVC + Repository Pattern

In cadrul backend-ului se aplica pattern-ul **MVC** (Model-View-Controller): modelele de domeniu reprezinta datele, controller-ele gestioneaza cererile HTTP, iar serviciile contin logica de business separata de accesul la date. Accesul la date este abstractizat prin **Repository Pattern**, fiecare entitate avand propriul repository.

Motivatie:

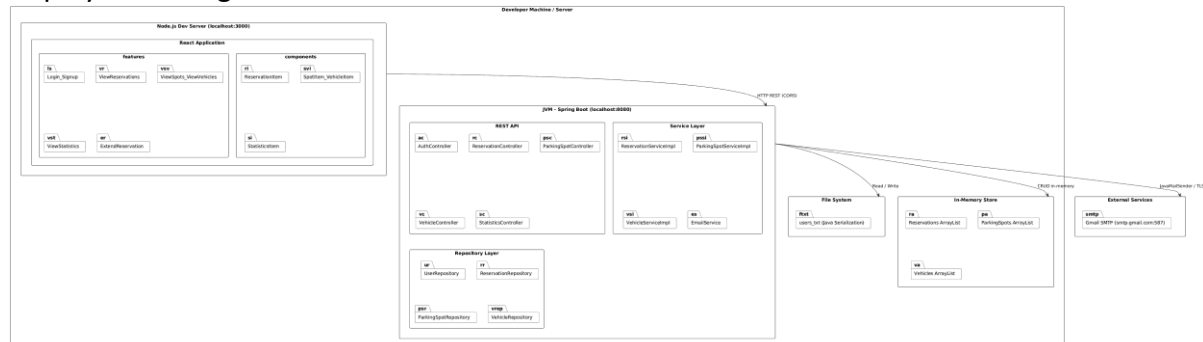
Alegerea arhitecturii pe niveluri este justificata de separarea clara a responsabilitatilor (Separation of Concerns), facilitand extinderea si testarea independenta a fiecarui strat. Comunicarea prin REST API permite inlocuirea frontend-ului fara modificari in backend. Repository Pattern permite schimbarea mecanismului de persistenta (ex. inlocuirea ArrayList cu o baza de date) fara a afecta logica de business.

Package Design

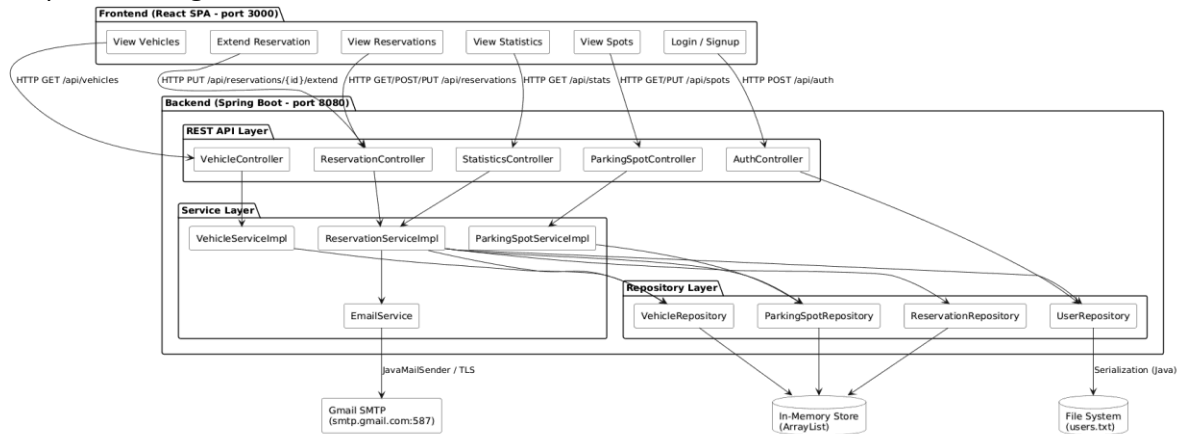


Component and Deployment Diagram

Deployment Diagram



Component Diagram

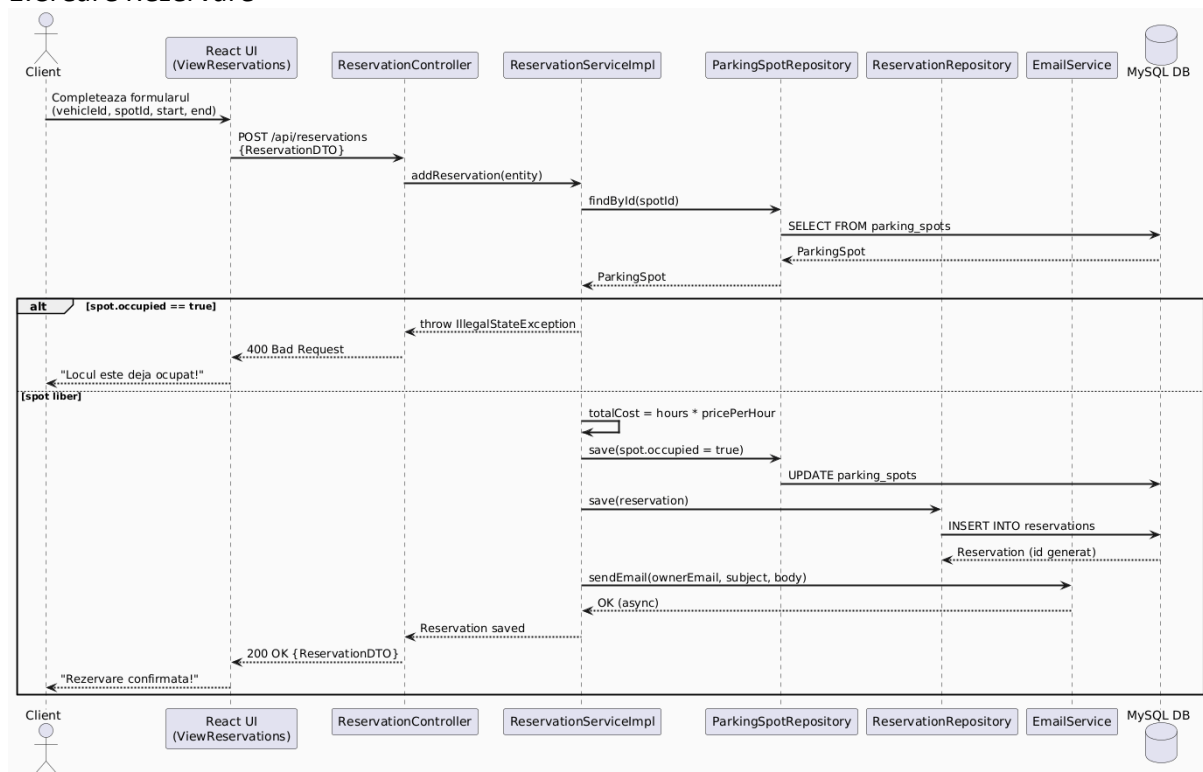


Deliverable 3

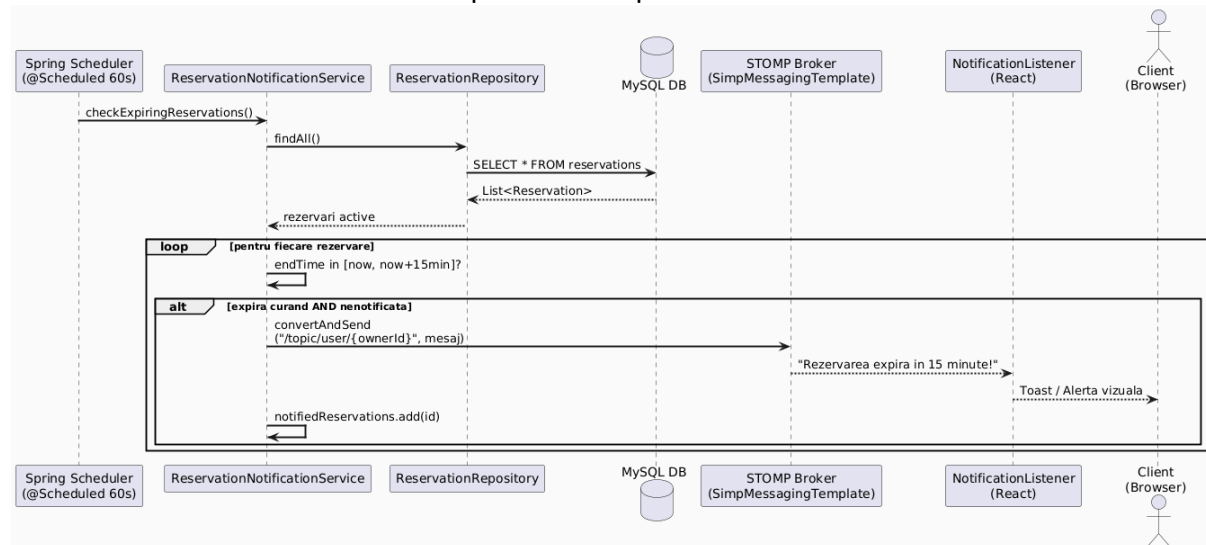
Design Model

Dynamic Behavior

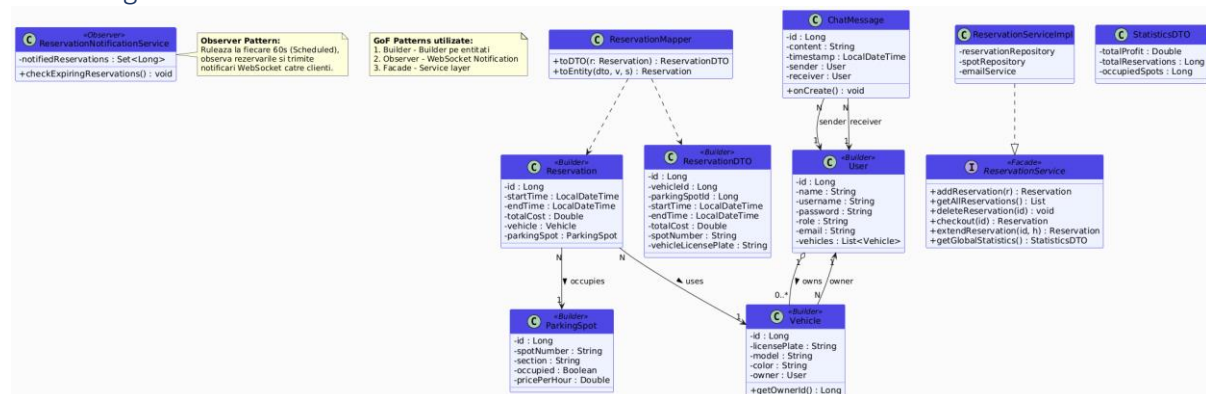
1. Creare Rezervare



2. Notificare WebSocket: Rezervare pe cale sa expire



Class Diagram



1. Builder Pattern Utilizat prin adnotatia Lombok `@Builder` pe clasele `User`, `Vehicle`, `ParkingSpot` si `Reservation`. Permite constructia obiectelor fara constructori cu multi parametri. Exemplu concret in `MockDataServiceImpl.java`:

java

```

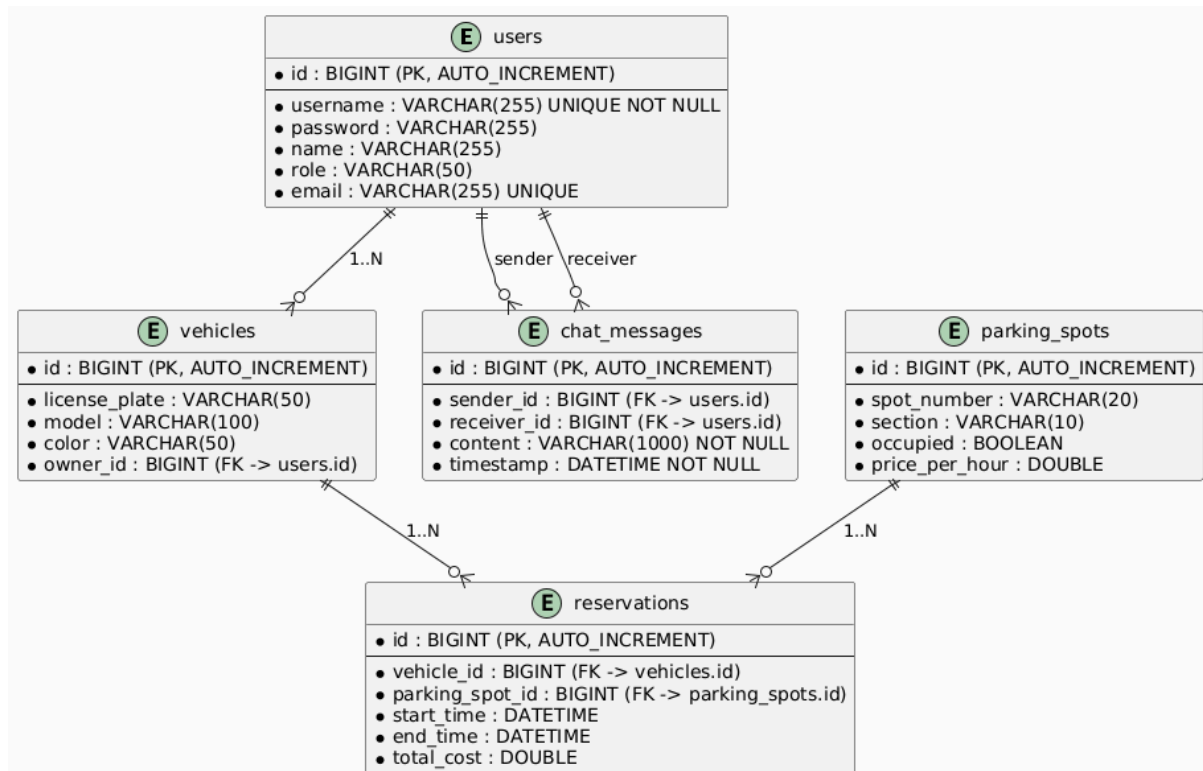
User admin = User.builder()
    .name("Administrator")
    .username("admin")
    .role("ROLE_ADMIN")
    .build();
    
```

2. Observer Pattern Implementat prin `ReservationNotificationService`. Aceasta clasa "observa" periodic (la fiecare 60 secunde, via `@Scheduled`) toate rezervarile din baza de date si, atunci cand detecteaza o rezervare care expira in mai putin de 15 minute, trimite automat o notificare prin WebSocket (STOMP) catre clientul vizat.

3. Facade Pattern Stratul de servicii

(`ReservationService`, `ParkingSpotService`, `VehicleService`) functioneaza ca o interfata simpla (Facade) intre controllere si repository-uri. Controllerele nu acceseaza niciodata direct repository-urile, ci numai prin servicii, ascunzand complexitatea logicii de business.

Data Model



System Testing

1. Testare manuala (Manual Testing) S-au verificat scenariile principale de utilizare direct in browser (React frontend) si prin Swagger UI (<http://localhost:8080/swagger-ui.html>) pentru endpoint-urile REST.

2. Testare prin Swagger / Postman (API Testing) Fiecare endpoint REST a fost testat individual, verificand codurile de raspuns HTTP, structura JSON returnata si comportamentul la input invalid.

Test Cases

TC-01: Autentificare cu date corecte Precondition: Userul andrei / parola exista in baza de date. Input: POST /api/auth/login cu body { username: "andrei", password: "parola" }

Expected result: 200 OK, se returneaza un UserDTO cu role: "ROLE_CLIENT". Result: PASS

TC-02: Autentificare cu parola gresita Input: POST /api/auth/login cu body { username: "andrei", password: "gresit" } Expected result: 401 Unauthorized. Result: PASS

TC-03: Creare rezervare pe loc liber Precondition: Locul de parcare cu parking_spot_id=1 are occupied=false. Input: POST /api/reservations cu vehicleId, spotId, startTime si endTime valide. Expected result: 200 OK, se returneaza ReservationDTO cu totalCost calculat corect; locul devine occupied=true in baza de date. Result: PASS

TC-04: Creare rezervare pe loc ocupat Precondition: Locul de parcare cu parking_spot_id=1 are occupied=true. Input: POST /api/reservations cu acelasi spotId. Expected result: 500 Internal Server Error cu mesajul "Locul de parcare este deja ocupat!". Result: PASS

TC-05: Extindere rezervare cu 2 ore Precondition: Exista o rezervare activa cu id=1. Input: POST /api/reservations/1/extend?extraHours=2 Expected result: 200 OK, endTime crescut

cu 2 ore, totalCost recalculat, email de confirmare trimis catre proprietarul vehiculului.
Result: PASS

TC-06: Checkout rezervare Precondition: Exista o rezervare activa cu id=1. Input: POST /api/reservations/1/checkout Expected result: 200 OK, endTime setat la momentul curent, totalCost recalculat proportional, locul de parcare devine occupied=false. Result: PASS

TC-07: Vizualizare statistici admin Precondition: Exista cel putin 2 rezervari in baza de date. Input: GET /api/reservations/statistics Expected result: 200 OK cu valorile corecte pentru totalProfit, totalReservations si occupiedSpots. Result: PASS

TC-08: Trimitere mesaj in chat Precondition: Exista un utilizator client si un utilizator admin in baza de date. Input: POST /api/chat/send cu body { senderId, receiverId, content } Expected result: 200 OK, mesajul este salvat in tabela chat_messages si retransmis prin WebSocket. Result: PASS

TC-09: Notificare WebSocket pentru rezervare pe cale sa expire Precondition: Exista o rezervare cu endTime in urmatoarele 15 minute. Input: Scheduler-ul ruleaza automat la interval de 60 de secunde. Expected result: Notificare trimisa pe canalul /topic/user/{id}, mesaj de alerta afisat in browser pentru utilizatorul respectiv. Result: PASS

TC-10: Inregistrare cu username deja existent Precondition: Username-ul "andrei" exista deja in baza de date. Input: POST /api/auth/register cu body { username: "andrei", ... } Expected result: 400 Bad Request cu mesajul "Username-ul exista deja!". Result: PASS

Future Improvements

- 1. Autentificare cu JWT (JSON Web Tokens)** In prezent, autentificarea este simpla (username + parola fara token). Implementarea JWT ar oferi securitate sporita, sesiuni stateless si suport pentru refresh tokens.
- 2. Sistem de plata online** Integrarea unui gateway de plata (ex. Stripe sau PayU) pentru plata rezervarilor direct din aplicatie, eliminand necesitatea platii cash la iesire.
- 3. Harta interactiva a parcarii** O reprezentare grafica a locurilor de parcare (ocupate/libere, pe sectiuni A si B), cu posibilitatea selectarii unui loc prin click direct pe harta.
- 4. Aplicatie mobila (React Native)** Extinderea frontendului web cu o aplicatie mobila cross-platform pentru iOS si Android, permitand utilizatorilor sa rezerve locuri de parcare de pe telefon.
- 5. Sistem de abonamente** Introducerea de abonamente lunare sau saptamanale pentru utilizatorii frecventi, cu preturi reduse fata de tariful orar standard.
- 6. Rapoarte si export date** Functionalitate pentru administratori de a exporta statisticile in format PDF sau CSV (ocuparea medie, profitul lunar, istoricul rezervarilor).
- 7. Integrare cu camere de supraveghere (ANPR)** Recunoasterea automata a placutelor de inmatriculare prin camere video la intrarea/iesirea din parcare, cu deschiderea automata a barierei pentru vehiculele cu rezervare activa.
- 8. Notificari push pe mobil** Trimiterea de notificari push (via Firebase Cloud Messaging) pe dispozitivele mobile ale utilizatorilor, nu doar prin WebSocket in browser.

Conclusion

Aplicatia a fost dezvoltata utilizand o arhitectura moderna pe trei niveluri: un frontend React.js (SPA), un backend Spring Boot cu API REST, si o baza de date MySQL gestionata prin JPA/Hibernate.

Pe parcursul dezvoltării au fost implementate funcționalitățile principale: autentificare bazată pe roluri (ROLE_ADMIN, ROLE_CLIENT), gestionarea rezervărilor cu calcul automat al costului, checkout și extindere, administrarea vehiculelor și a locurilor de parcare, un sistem de notificări în timp real prin WebSocket (STOMP) și un modul de chat între client și administrator.

Din punct de vedere al design-ului software, au fost aplicate trei tipare GoF: **Builder** (construcția obiectelor de domeniu), **Observer** (monitorizarea rezervărilor și notificarea automată) și **Facade** (stratul de servicii ca interfață unificată pentru logica de business).

Bibliography

1. Spring Boot Documentation (2024). *Spring Framework Reference Documentation*.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
2. React Documentation (2024). *React – A JavaScript library for building user interfaces*.
<https://react.dev/>
3. STOMP Protocol (2024). *Simple Text Oriented Messaging Protocol*.
<https://stomp.github.io/>
4. PlantUML Documentation (2024). *PlantUML Language Reference Guide*.
<https://plantuml.com/guide>